command. Pi will run this program at bootup, and before other services are started. Pi will not be able to complete its boot process if the program is not concluded with an ampersand. The ampersand allows the command to run in a separate process and continue booting with the main process running. Use the sudo reboot command to reboot the Pi to test it.

If you add a script into /etc/rc.local, it will be added to the boot sequence. So be careful as to which code is being run at boot and test the code when necessary. You can also get the script's output and error written to a text file (say log.txt) and use it to debug.

```
• sudo python /home/pi/sample.py & > /home/pi/Desktop/log.
txt 2>&1
```

Users can also run a file runtime by using the .bashrc file. With the .bashrc method, it will now run on boot and whenever the terminal is opened or a new secure shell client is established. The command '/home/pi/.bashrc' should be inserted at the bottom of the notepad file. sudo nano /home/pi/.bashrc

```
• echo Running at boot
• sudo python /home/pi/sample.py
```

Run the following command and observe the unique way in which the system responds to the conditional inputs and formats:-

```
• # A first Python Script, playing around with strings,
loops, and conditions.
• print('This is my first script') # and this is a comment
• string1 = 'hello'
• string2 = 'world'
• # strings can be concatenated with + and repeated with *
• print(string1 + ' ' + string2 + '!'*3 )
• twoOnThree = 2/3
• print('2 divided by 3 is about {}' .format(twoOnThree)) #
you can print variables
• print('or with fewer decimal points: {: f}'
.format(twoOnThree)) # you can explicitly format the output
• import math #Usually we put imports at the start. I put
it here for context.
• string3 = 'more'
• print('you can print {0} than one variable, here\'s Pi:
{1}. That makes {2} variables!' .format(string3,math.pi,3))
```